

GUIA QUICKSTART · PWA PARA APK

De PWA para **APK** Android

3 métodos comparados.
1 recomendado.

Passo a passo para transformar a PWA **MirrorTV** em um APK instalável no Android — desde a publicação na **Vercel** até o download do pacote no **PWABuilder** e a instalação no celular. Sem Android Studio, sem código nativo, em menos de 30 minutos.

30 min

TEMPO TOTAL

\$0

CUSTO

3

MÉTODOS

VERSÃO

1.0 · 2026

PLATAFORMA

Android 9+

PÁGINA

Quickstart

● INSTALÁVEL COMO APP NATIVO

1. Visão Geral

Converter uma **PWA** (Progressive Web App) em um **APK** (Android Package) significa pegar o app MirrorTV que roda no navegador e empacotá-lo como um aplicativo nativo Android, instalável diretamente pelo arquivo `.apk` e distribuível fora da Play Store. O Android suporta esse fluxo através de uma tecnologia chamada **TWA (Trusted Web Activity)**, que abre a PWA em uma WebView de tela cheia, sem a barra de endereço do Chrome, mas herdando todos os recursos do navegador — incluindo o `getDisplayMedia` usado pelo MirrorTV para capturar a tela.

Existem três caminhos para chegar ao APK, todos legítimos e gratuitos. Este guia compara os três e depois detalha o **método recomendado para 90% dos casos**: publicar a PWA na Vercel e empacotar com o **PWABuilder**. Você não precisa instalar Android Studio nem escrever uma linha de Java/Kotlin — o tempo total do início ao fim é inferior a 30 minutos.

Pré-requisitos

Recurso	Por quê	Versão mínima
Computador com Chrome	PWABuilder exige Chrome desktop para análise	Chrome 110+
Conta GitHub	Vercel deploys a partir de um repositório Git	Gratuita
Conta Vercel	Hospedagem HTTPS gratuita para a PWA	Gratuita (Hobby)
Node.js + Bun (opcional)	Apenas se for rodar o projeto localmente antes do deploy	Node 20+, Bun 1.3+
Celular Android para teste	Onde instalar e validar o APK gerado	Android 9 (API 28)+

Tabela 1 — Pré-requisitos do quickstart

2. Comparação dos 3 Métodos

Todos os três métodos chegam ao mesmo resultado — um APK baseado em TWA que abre a sua PWA em tela cheia. A diferença está no **esforço**, no **controle** e na **automação**. A tabela abaixo resume quando usar cada um; em seguida, três callouts destacam os prós e contras.

Tabela comparativa

Critério	PWABuilder	Bubblewrap CLI	Android Studio TWA
Dificuldade	Muito fácil	Intermediária	Avançada
Tempo até APK	5 min	15 min	45 min+
Instalação no PC	Nenhuma (web)	Node + Bubblewrap	Android Studio (~3 GB)
Customização	Básica (ícone, cor)	Média (manifest, build.gradle)	Total (código nativo)
Assinatura do APK	Automática (debug)	Configurável (keystore)	Manual (Android Studio)
Melhor para	Primeiro APK, prova de conceito	Recompilar a cada release	Adicionar código nativo
Play Store (AAB)	Sim, gera AAB	Sim, gera AAB	Sim, gera AAB

Tabela 2 — Comparação dos três métodos de empacotamento

Prós e contras em detalhe

PWABuilder — o caminho mais curto

Prós: tudo acontece no navegador, sem instalar SDK; gera APK e AAB prontos; integração direta com a Play Store. Contras: pouca customização visual além do ícone e cor de splash; assinatura de produção precisa ser gerada manualmente uma vez (upload de keystore). **Recomendado para 90% dos casos.**

Bubblewrap CLI — para quem recompila sempre

Prós: roda local, integra com CI/CD (GitHub Actions), permite ajustar `build.gradle` e `twamainfest.json`. Contras: exige Node 16+ e JDK 11+; primeira configuração demora ~15 min; curva de aprendizado da CLI. Ideal depois que você já validou o APK pelo PWABuilder e quer automatizar releases.

Android Studio + TWA — máximo controle

Prós: abre o projeto Android completo, permite adicionar código nativo (permissões, intents, plugins), editar layouts XML. Contras: download de ~3 GB, JDK configurado, tempo de build elevado; só vale a pena se a PWA precisar de APIs que o TWA não expõe por padrão (raro). **Pule este método** a menos que tenha um motivo concreto.

Decisão: este guia detalha o **PWABuilder** a partir da próxima seção. Se depois você quiser migrar para Bubblewrap, o comando `bubblewrap init --manifest https://sua-url/manifest.json` reaproveita o mesmo `manifest.json` da PWA — a migração é direta, sem retrabalho.

3. Passo 1 — Publicar a PWA na Vercel

O PWABuilder precisa de uma **URL pública com HTTPS** para analisar a PWA e gerar o APK. A Vercel é a forma mais rápida de publicar um projeto Next.js: em poucos cliques você tem um endereço como `https://mirrortv.vercel.app` com HTTPS automático, CDN global e renovação de certificado gerenciada.

1

Suba o projeto MirrorTV para o GitHub

Crie um repositório no GitHub (público ou privado, tanto faz) e faça push do projeto Next.js. Inclua o arquivo `package.json`, a pasta `src/`, `public/` e `next.config.ts`. Não suba `node_modules/` nem `.next/` — o arquivo `.gitignore` do scaffold já cobre isso.

BASH

```
# No diretório do projeto
git init
git add .
git commit -m "MirrorTV PWA inicial"
git branch -M main
git remote add origin https://github.com/SEU_USUARIO/mirrortv.git
git push -u origin main
```

2

Conecte a Vercel ao repositório

Acesse `vercel.com/new`, faça login com o GitHub e clique em **Import Project** no repositório `mirrortv`. A Vercel detecta Next.js automaticamente e preenche **Build Command** (`next build`) e **Output Directory** (`.next`) sem intervenção.

3

Configure as variáveis de ambiente (se houver)

Se o projeto usa variáveis de ambiente (chaves de API, endpoints de mini-services), adicione em **Settings → Environment Variables**. Para o MirrorTV não há segredos obrigatórios — o serviço de signaling e o DLNA rodam em mini-services à parte, que você pode subir no Railway, Fly.io ou outra VPS.

4

Deploy e obtenha a URL pública

Clique em **Deploy**. Em 60–90 segundos a Vercel publica o projeto e retorna a URL `https://mirrortv.vercel.app` (ou o nome que você escolheu). Abra a URL no Chrome desktop para confirmar que o app carrega, que o ícone aparece na aba e que o Chrome exibe o ícone de instalação na barra de endereço — sinal de que a PWA está válida.

BASH

```
# Validação rápida da PWA via CLI (opcional)
npx pwa-asset-generator --manifest \
  https://mirrortv.vercel.app/manifest.json

# Ou use o Lighthouse no Chrome DevTools
# Abra a URL → F12 → Lighthouse → Generate report
# Score PWA deve ser >= 90
```

Por que HTTPS é obrigatório?

As APIs de captura de tela (`getDisplayMedia`), Service Worker e instalação de PWA só funcionam em HTTPS (ou em `localhost` para desenvolvimento). Sem HTTPS, o PWABuilder rejeita a URL e o APK gerado não consegue capturar a tela. A Vercel resolve isso automaticamente — você não precisa configurar certificados.

4. Passo 2 — Gerar o APK no PWABuilder

O **PWABuilder** é uma ferramenta online da Microsoft que analisa a PWA, valida o `manifest.json`, gera ícones faltantes e empacota tudo em um APK/AAB assinável. Não pede login, não cobra nada e funciona em qualquer navegador Chromium.

1

Abra pwabuilder.com no Chrome desktop

Acesse <https://www.pwabuilder.com>. Use Chrome, Edge ou Brave — Firefox e Safari não têm suporte completo às APIs de empacotamento que o site usa internamente.

2

Cole a URL da Vercel e analise

Na caixa central, cole <https://mirror.tv.vercel.app> e clique em **Start**. O PWABuilder baixa a PWA, valida o manifest, os service workers e os ícones. Em 30 segundos você recebe um **score** (procure ficar acima de 90) e uma lista de melhorias opcionais. Se algo crítico faltar (ícone 512×512, `start_url`, `display: standalone`), ele aponta exatamente o que corrigir no `manifest.json`.

3

Package For Stores → Android

No painel de resultado, clique em **Package For Stores** e selecione **Android**. Um formulário pede seis campos. Preencha conforme a tabela abaixo e clique em **Generate**.

Campo	Exemplo / valor recomendado	Observação
Package ID	com.seunome.mirror.tv	Único global; reverse-domain
App name	MirrorTV	Aparece sob o ícone no launcher
Launcher name	MirrorTV	Versão curta (≤ 12 chars)
App version	1.0.0	Semântico (maior.minor.patch)
Signing key	Gerar nova (padrão)	Guarde a senha! Necessária p/ updates
Display mode	standalone	Tela cheia sem barra do Chrome

Tabela 3 — Campos do formulário Android do PWABuilder

4

Baixe o ZIP com o APK

Após ~1 minuto o PWABuilder entrega um arquivo `mirror.tv-android-signed.zip`. Dentro dele há: **(a)** o `.apk` pronto para sideload, **(b)** o `.aab` (Android App Bundle) para Play Store, **(c)** o `.keystore` com que o APK foi assinado — **guarde esse keystore**, sem ele não dá para publicar updates.

TEXT

```
mirror.tv-android-signed.zip
├─ MirrorTV.apk           ← instalar no celular
├─ MirrorTV.aab           ← publicar na Play Store
├─ signing.keystore       ← GUARDAR (senha do passo 3)
└─ README.txt             ← instruções de assinatura
```

5. Passo 3 — Instalar o APK no Android

Com o `.apk` assinado em mãos, o último passo é o **sideload**: instalar manualmente sem passar pela Play Store. Esse fluxo é ideal para teste interno, distribuição para poucos usuários ou prova de conceito.

1

Transfira o APK para o celular

Use o método mais prático: Google Drive, Dropbox, Telegram (envie para si mesmo), ou cabo USB. O importante é que o arquivo `MirrorTV.apk` esteja acessível pelo gerenciador de arquivos do Android.

2

Habilite 'Fontes desconhecidas'

No Android 9+, a permissão é por-app: vá em **Configurações → Aplicativos → Acesso especial → Instalar apps desconhecidos** e ative o gerenciador de arquivos que você vai usar (Files, Solid Explorer, etc.). No Android 8 e anteriores a opção fica em **Configurações → Segurança → Fontes desconhecidas**.

3

Toque no APK e instale

Abra o APK pelo gerenciador de arquivos. O Android mostra um resumo de permissões (internet, captura de tela, áudio) e um botão **Instalar**. Toque e aguarde 10–20 segundos. Ao final, escolha **Abrir** ou **Concluir** — o ícone do MirrorTV estará na tela inicial, junto dos outros apps.

4

Valide o espelhamento

Abra o MirrorTV pelo ícone (não mais pelo Chrome). Vá até a aba **Chromecast**, toque em **Selecionar TV**, escolha a Smart TV da mesma rede Wi-Fi e autorize a captura de tela. O espelho deve começar em 5–10 segundos. Se funcionar, está pronto.

Aviso sobre Google Play Protect

O Play Protect pode exibir um alerta ('aplicativo não reconhecido') ao instalar APKs fora da Play Store. Isso é esperado e **não bloqueia a instalação** — basta tocar em 'Instalar mesmo assim'. Como o APK é seu, assinado por você, o risco é zero. Para distribuição pública, use a Play Store (ver próxima seção).

6. Próximos Passos

Com o APK funcionando, três caminhos naturais de evolução. Escolha conforme seu objetivo — distribuição ampla, identidade visual refinada, ou automação de release.

Trilha	O que envolve	Quando fazer
Publicar na Play Store	1. Criar conta de desenvolvedor Google (\$25, taxa única) 2. No PWABuilder, baixar o .aab (não o APK) 3. Criar listing no Play Console: nome, descrição, screenshots, ícone 512×512, banner 1024×500 4. Fazer upload do AAB em 'Produção' e enviar para revisão	Quando o app estiver pronto para uso público
Customizar ícone e splash	1. Substituir <code>public/icon-512.png</code> e <code>icon-maskable-512.png</code> pela marca própria 2. No <code>manifest.json</code>, ajustar <code>theme_color</code> e <code>background_color</code> 3. Redeploy na Vercel e reempacotar no PWABuilder	Antes de publicar — define a identidade visual
Automatizar releases (CI/CD)	1. Instalar Bubblewrap CLI: <code>npm i -g @bubblewrap/cli</code> 2. Inicializar: <code>bubblewrap init --manifest https://mirrortv.vercel.app/manifest.json</code> 3. Build: <code>bubblewrap build</code> → gera APK/AAB local 4. Integrar em GitHub Action: rodar a cada tag	Quando você tiver 3+ releases manuais e quiser automatizar

Tabela 4 — Trilhas de evolução pós-APK

Atualizar versões depois

Sempre que você alterar o código e fizer um novo deploy na Vercel, a PWA publicada é atualizada automaticamente — não é preciso reempacotar o APK. O Android checa o service worker ao abrir o app e baixa a versão nova em background. Para mudanças no próprio APK (nome, ícone, versão interna), refaça o passo 4 do PWABuilder com a nova versão (1.0.1, 1.1.0...) e reinstale. O Android entende como update se o **Package ID** for o mesmo e a **versão** for maior — caso contrário, instala como app novo lado a lado.